procgen_rl.py

PROCGEN RL

Teaching an agent to play levels it has never seen

```
$ python train.py --hours 300
> generating levels ..... ok
> training the agent (PPO) ..... ok
> explaining its decisions ..... ok
> racing the ghosts ..... ok
```

[Your Name] // [Course] // October 2026

AGENDA

```
> ls talk/
```

01 the problem: a game it can't memorise

02 what the agent sees: its 847 inputs

03 the model: network and one decision

04 how it learns: reward and PPO

05 opening the black box: debug mode

06 results and generalisation

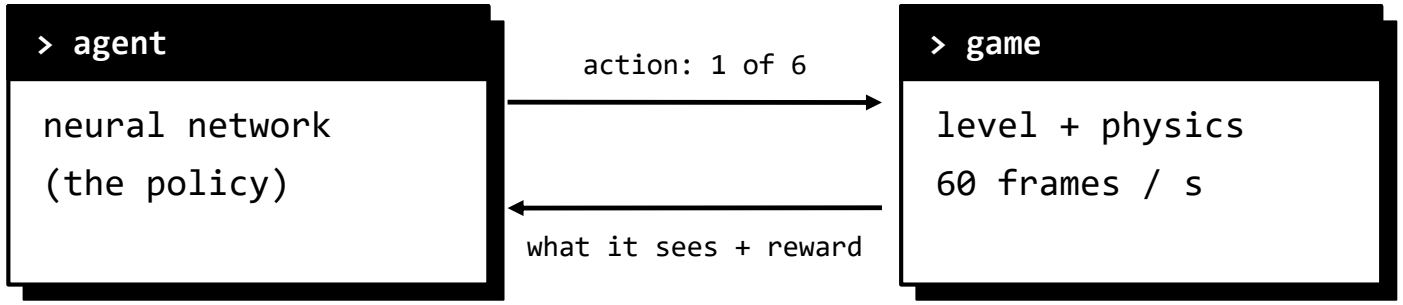
07 live demo: race the ghosts

> cd section_01

01

THE PROBLEM

RL IN ONE SLIDE



- > **agent** the player the network controls
- > **observation** what it sees: 847 numbers
- > **action** one of 6 button combinations
- > **reward** points: + progress, - dying, + flag
- > **episode** one attempt at one level
- > **policy** the rule from observation to action

> in plain words

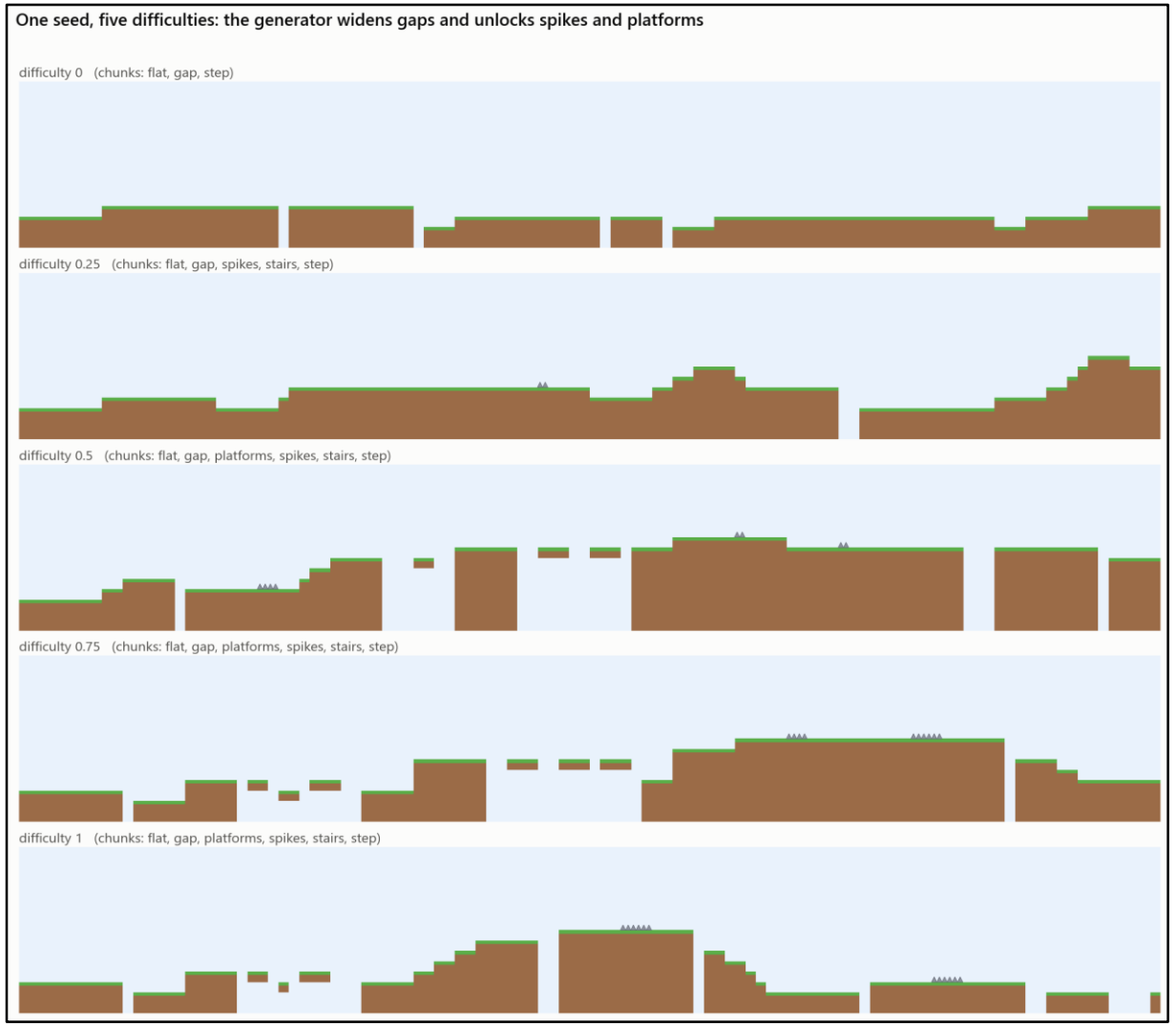
Nobody tells the agent how to play.

It starts by pressing random buttons. Actions that led to more points become more likely; actions that led to dying become less likely.

Repeat 16 million times.

trial and error, at scale

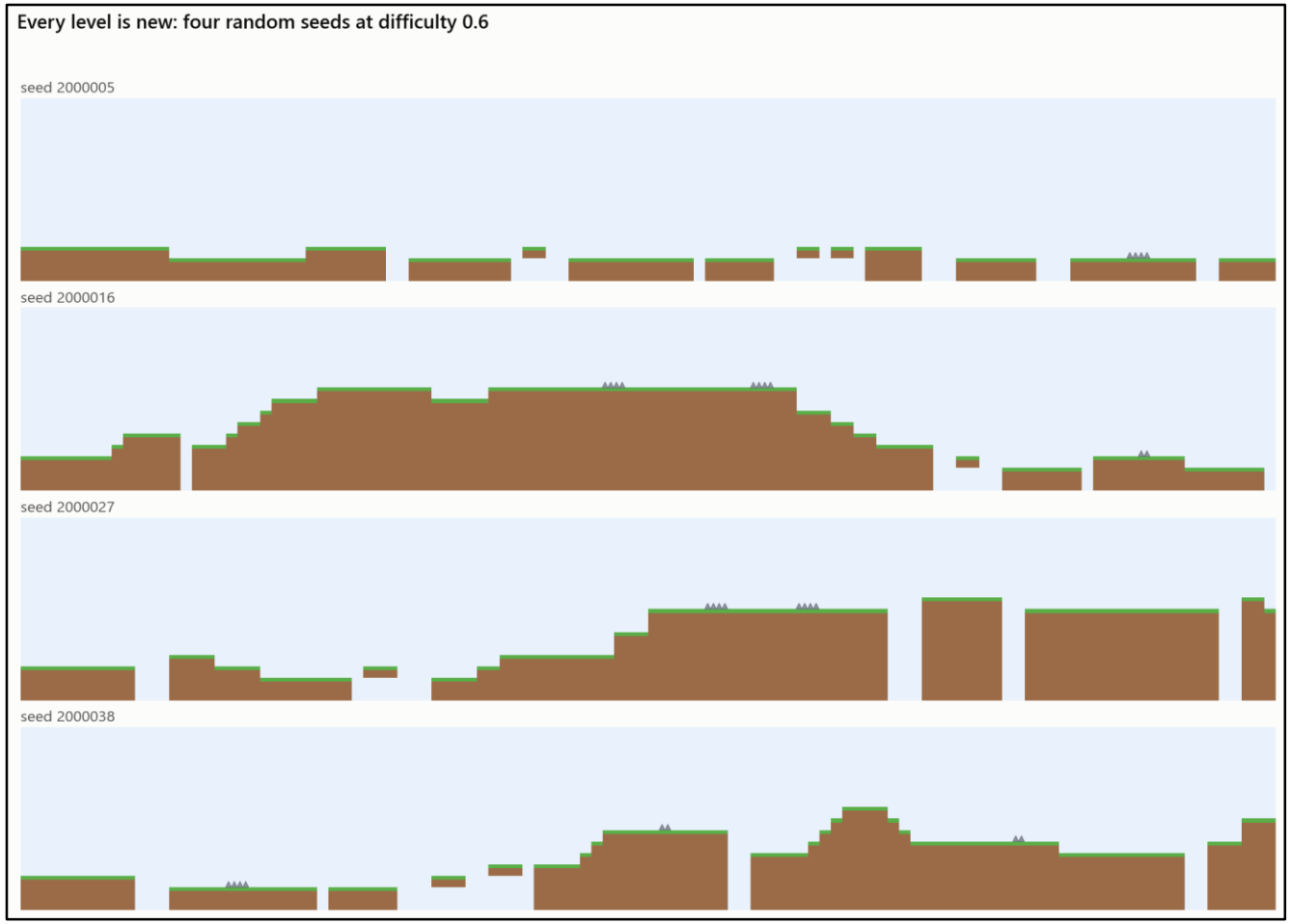
THE GAME



one seed at five difficulties

- **Goal:** run right and touch the flag
- **Terrain:** flat ground, steps, stairs, gaps
- **Hazards:** spikes from difficulty 0.25, floating platforms from 0.35
- **Physics:** 60 frames/s, jumps up to 3.8 tiles, hold jump to go higher
- **Difficulty:** one number from 0 to 1 widens gaps and adds hazards (the demo goes up to 2)

EVERY LEVEL IS NEW



1,000,000

training seeds: a fresh level for every attempt

100%

of tested levels solvable: a search solver beats every one

3 ranges

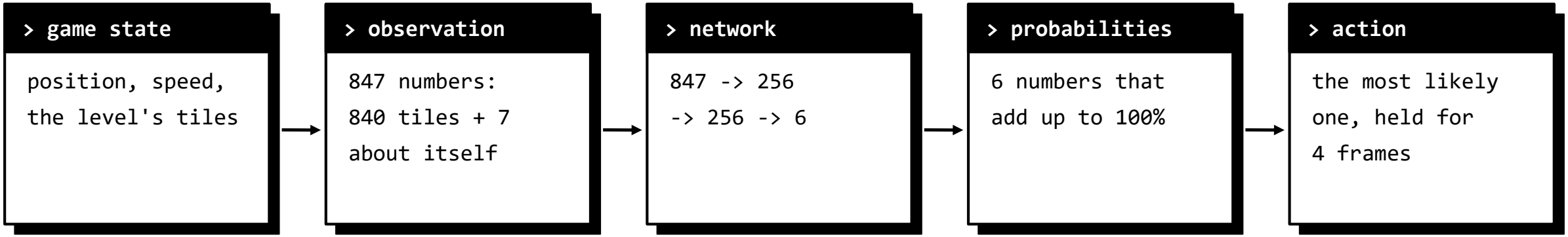
of seeds that never overlap: training, testing, demo

> cd section_02

02

WHAT THE AGENT SEES

ONE DECISION, START TO END



15 / s

decisions per second of play (every 4th frame)

< 1 ms

per decision, plain NumPy, no GPU

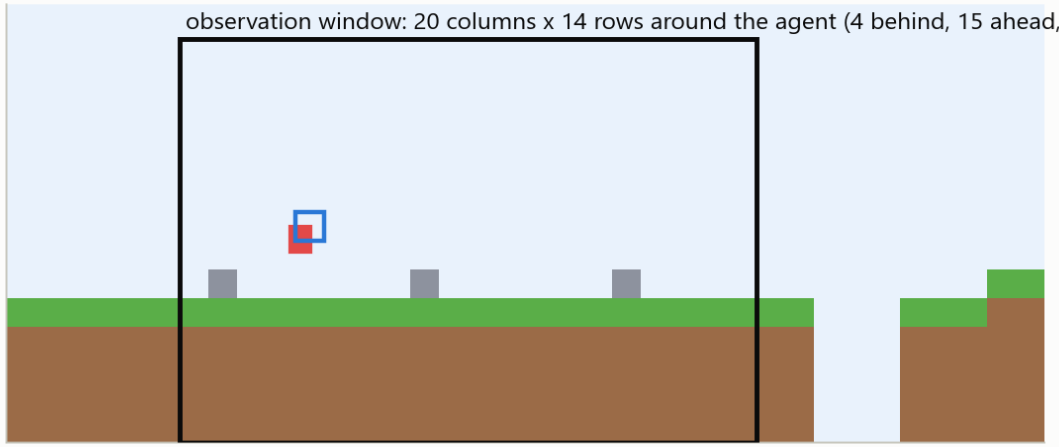
1 code path

the same stepping code in training, testing and the demo

frame 0: decide -> frames 0-3: hold the buttons -> frame 4: decide again

INPUT 1: THE TILE WINDOW

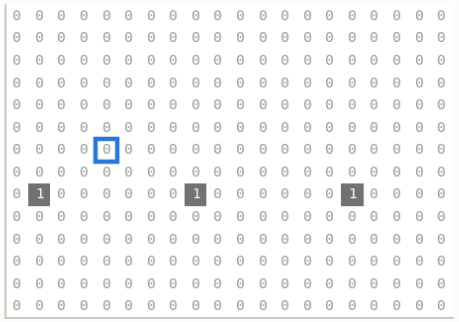
One real moment: seed 1000500, difficulty 0.6, decision 76



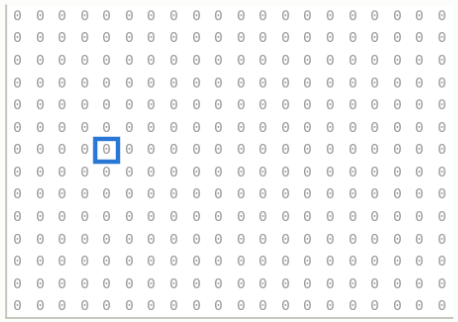
layer 1: solid (inputs 1-280)



layer 2: spike (inputs 281-560)



layer 3: goal (inputs 561-840)



inputs 841-847: about itself

841	vx / run speed	+1.00
842	vy / max fall	+0.56
843	on_ground	+0.00
844	can_jump	+0.00
845	jumping	+0.00
846	x within tile	+0.17
847	y within tile	+0.95

blue square = the agent's own tile
1 = yes, 0 = no for every tile

A real moment of the 300h ghost (seed 1000500, difficulty 0.6). Grey squares in the level are spikes; the blue square is the agent's own tile.

HOW A TILE BECOMES NUMBERS

tile	solid	spike	goal
air	0	0	0
ground / platform	1	0	0
spike	0	1	0
outside the level (wall)	1	0	0
air past the finish line	0	0	1

> numbering

```
input = layer x 280 + row x 20
        + column + 1

layer   0 solid, 1 spike, 2 goal
row     0 (6 up) .. 13 (7 down)
column  0 (4 behind) .. 19 (15 ahead)
```

spike 2 tiles ahead, at foot level
= 1x280 + 6x20 + 6 + 1 = **input 407**

Why 3 yes/no layers and not one number per tile?
"air = 0, solid = 1, spike = 2" would tell the network a spike is "twice a solid". Separate yes/no layers carry no fake ordering (one-hot encoding).

INPUT 2: 7 NUMBERS ABOUT ITSELF

#	name	computed as	range	why it needs it	example
841	vx	run speed / 0.15	-1 .. 1	how far a jump will carry	+1.00
842	vy	vertical speed / 0.5 (+ = down)	-0.8 .. 1	rising or falling	+0.56
843	on_ground	1 if standing on something	0 / 1	can it jump right now	+0.00
844	can_jump	1 up to 5 frames after a ledge	0 / 1	late jumps still work	+0.00
845	jumping	1 while rising with jump held	0 / 1	letting go cuts the jump	+0.00
846	x in tile	position inside its tile	0 .. 1	exact take-off point	+0.17
847	y in tile	position inside its tile	0 .. 1	exact height	+0.95

example = the moment on the previous slides: running at full speed, in the air, falling

all 847 inputs are between -1 and 1: neural networks learn best on small numbers

HOW THE NETWORK READS 847 NUMBERS



one flat list of 847 numbers: the network's whole world

> no picture, just a list

The network never sees an image. Input 407 always means "spike, 2 tiles ahead, at foot level". Meaning comes only from the position in the list.

> relative to the agent

The window moves with the agent, so a gap 3 tiles ahead gives the same numbers anywhere, in any level. That is why it can play levels it has never seen.

> nothing hand-coded

No rule says "jump before a gap". Each first-layer neuron gives every input a learned weight; training finds weights that turn tile patterns into the right button.

> small on purpose

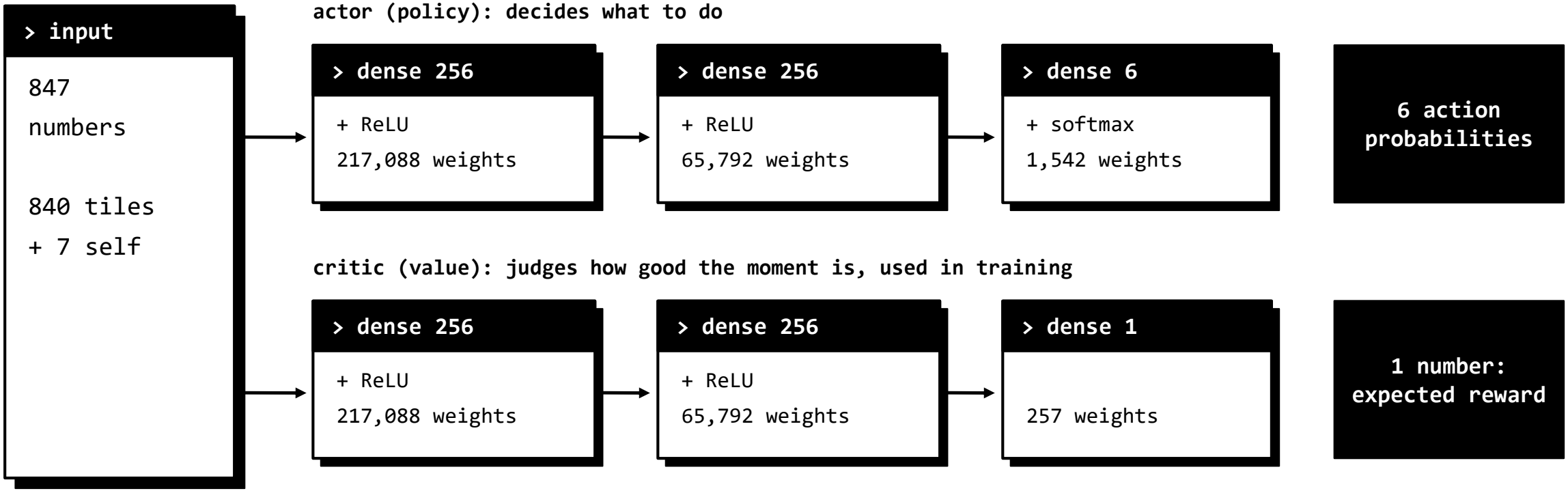
847 numbers instead of about 100,000 pixels, so a small fully connected network is enough and it trains on a laptop CPU.

> cd section_03

03

THE MODEL

ARCHITECTURE



284,422
actor parameters

283,137
critic parameters

ReLU MLP
no convolution, no memory between decisions

2.3 MB
one ghost file (NumPy weights)

WHAT ONE LAYER COMPUTES

> 1 weighted sum

`z = W x + b`

Each of 256 neurons multiplies all 847 inputs by its own weight, adds them up and adds a bias.

= "how much does this view look like my pattern?"

> 2 ReLU

`h = max(0, z)`

Negative results become 0: a neuron stays silent unless its pattern is there.

In the example, 57 of 256 neurons in layer 1 fire.

> 3 softmax

`p = e^z / sum(e^z)`

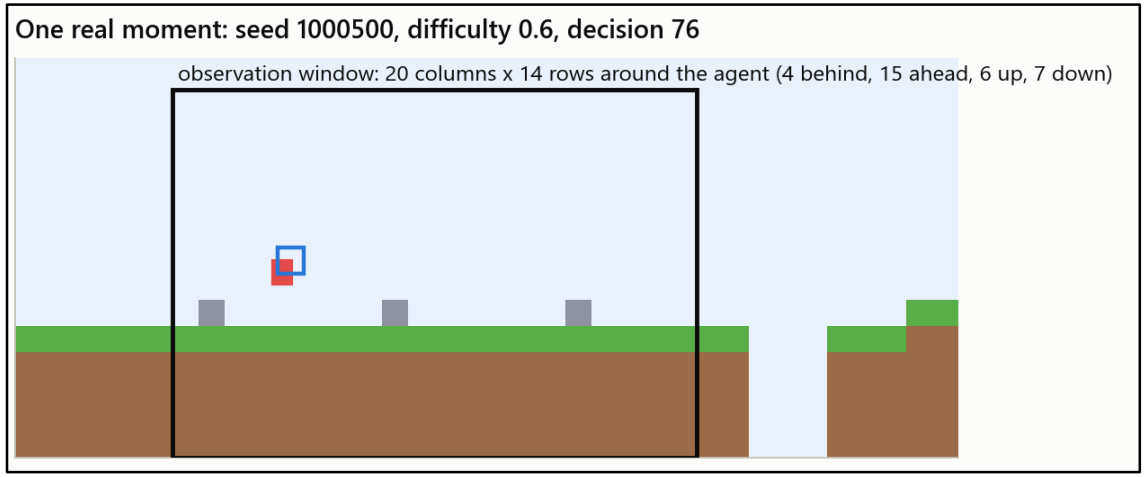
Turns the 6 output scores into probabilities that add up to 100%.

The demo takes the largest; training samples from them to explore.

layer 1 finds tile patterns -> layer 2 combines them into situations -> output scores each button

W and b are the 567,559 numbers training adjusts; everything else is fixed

A REAL FORWARD PASS



> **input:** 847 numbers, 103 of them are 1

> **hidden 1:** 57 of 256 neurons fire

> **hidden 2:** 16 of 256 neurons fire

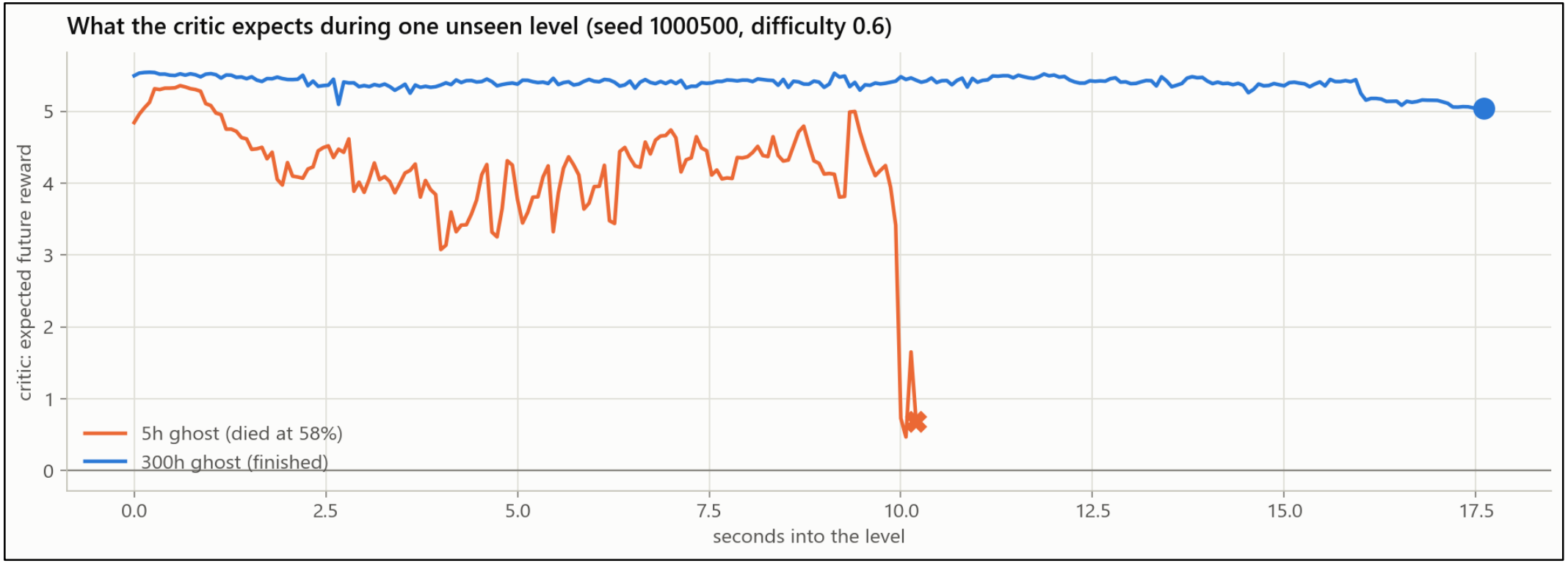
> **critic:** expects +5.43 more reward

action	score (logit)	probability
idle	3.12	19.1%
left	-2.66	0.1%
> right	3.95	44.0%
jump	1.53	3.9%
left+jump	-2.05	0.1%
right+jump	3.66	32.9%

> reading it

It is in the air and falling, so "right" (44%) and "right+jump" (33%) do almost the same thing. The demo takes the top one: **right**.

THE CRITIC: HOW IT JUDGES A MOMENT



> what

A second network, same shape, one output: how much reward it expects from here on.

> why

Training compares what really happened with this guess. Better than expected: do that more.

> what it shows

5h ghost: 5.3 falls to 0.5 as it drops into a pit. 300h: stays above 5.0.

> cd section_04

04

HOW IT LEARNS

REWARD: THE ONLY FEEDBACK

> reward

+0.1

for every new tile of progress to the right

> reward

-0.002

every decision: hurry up

> reward

-1

for dying: falling or touching a spike

> reward

+5

for touching the flag

> an attempt ends when

it dies or reaches the flag

6 s pass without new progress

100 s pass in total

> why reward progress

A random agent almost never reaches the flag. Points for every new tile give feedback from the very first second; moving back and forth earns nothing.

PPO, STEP BY STEP

> 1 play

8 copies of the game, 256 decisions each: 2,048 decisions with the current network. It samples actions, so it explores.

> 2 score

For each decision: advantage = what really followed (rewards, discounted by 0.99 per step) minus what the critic expected.

> 3 update

Positive advantage: make that action more likely; negative: less. Clip: no probability moves more than 20% per round. 5 passes, batches of 512.

> 4 repeat

7,911 rounds for 300 h of play. The critic is also trained to predict the rewards better.

> why PPO

Proximal Policy Optimization (Schulman et al., 2017).

The standard, stable choice for games like this.

The clip stops one lucky or unlucky batch from wrecking what was learned.

Implementation: Stable-Baselines3.

TRAINING SETUP

setting	value
algorithm	PPO (Stable-Baselines3)
network	2 x 256 ReLU, actor and critic separate
parallel games	8
decisions per game per round	256
batch size / passes	512 / 5
learning rate	0.0003
discount gamma / GAE lambda	0.99 / 0.95
clip range	0.2
entropy bonus (keep exploring)	0.01

300 h

of simulated play = 16.2 million decisions

4.7 h

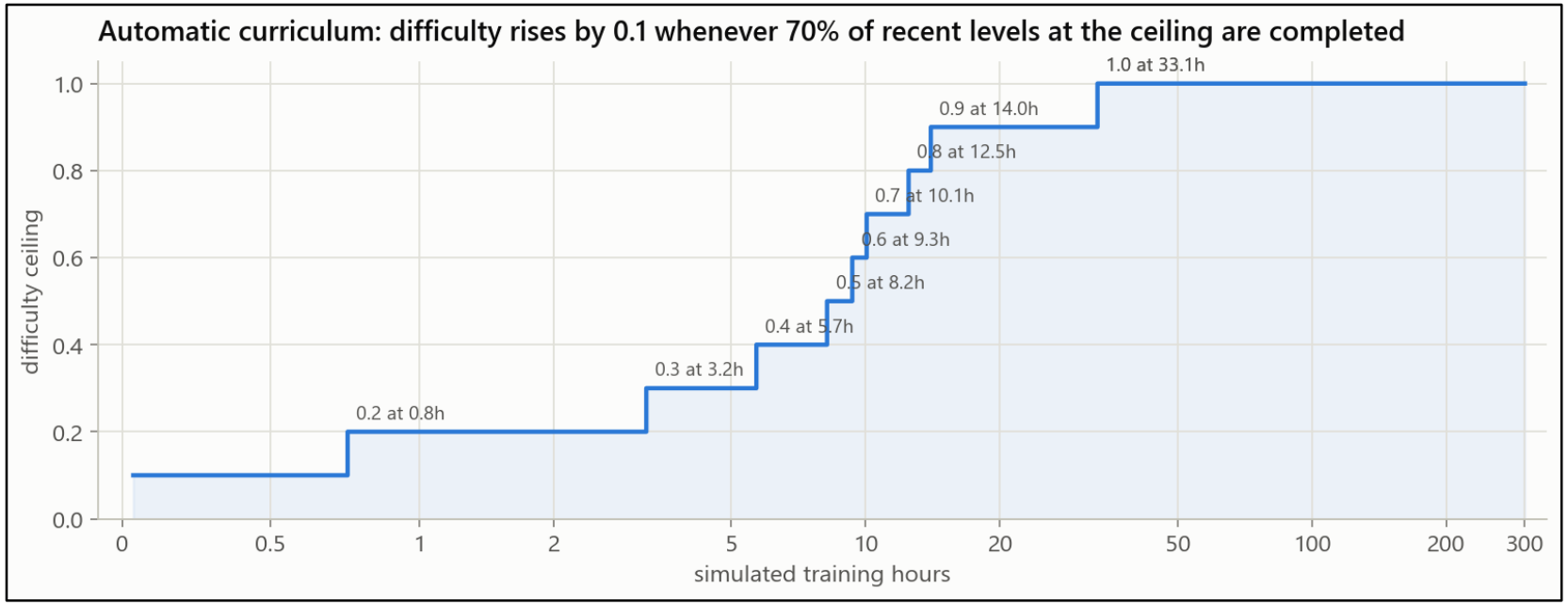
of real time on a laptop CPU, no GPU

~2,700 / s

decisions: about 180x faster than real time

"300 h" = 16.2 M decisions / 15 per second: practice time summed over the 8 parallel games

A CURRICULUM FROM EASY TO HARD



0.1 -> 0.9

difficulty in the first 14 simulated hours

1.0 at 33 h

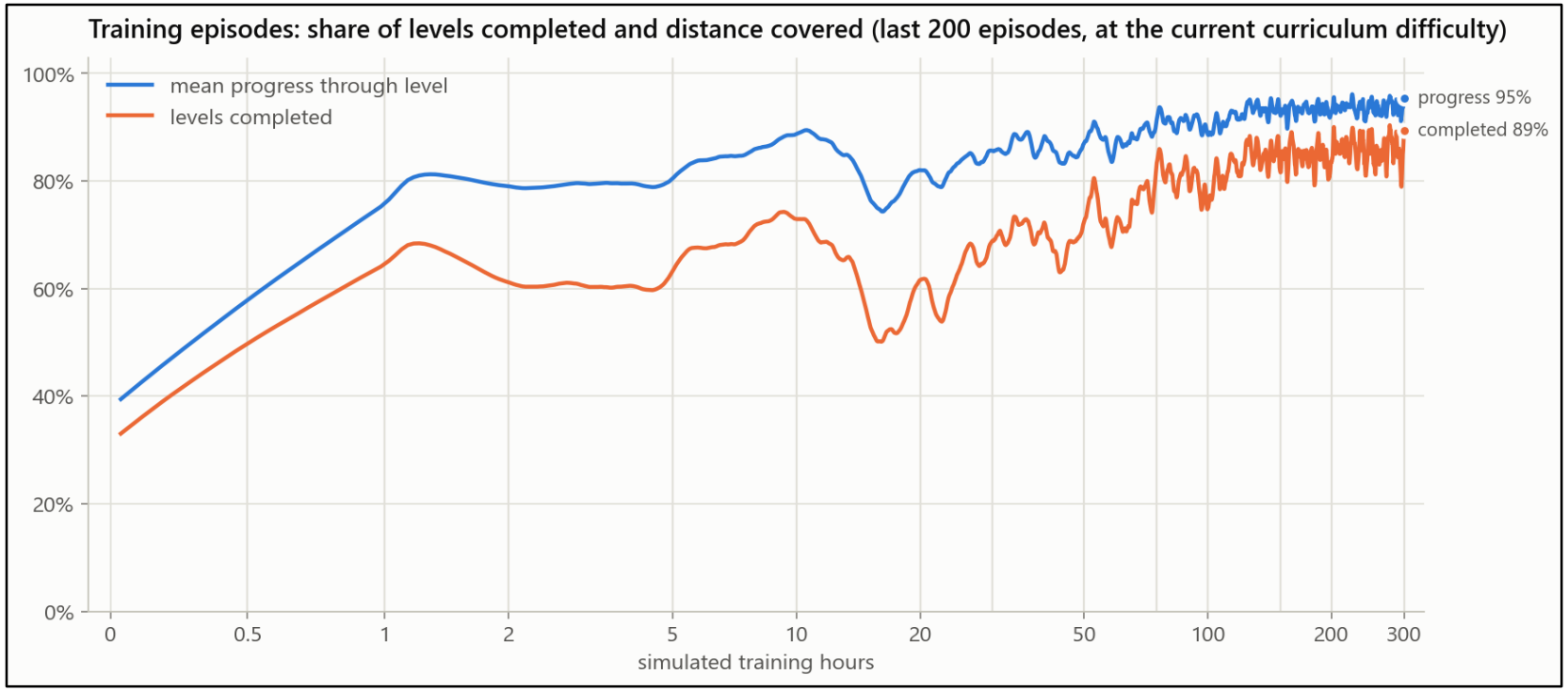
the last step, the widest gaps, was the hardest

70%

of recent levels completed: move up by 0.1

70% of levels at the current ceiling, 30% easier ones, so old skills are not forgotten

THE LEARNING CURVE



33% -> 89%

of training levels completed, first to last

dips

= the curriculum making levels harder

log axis

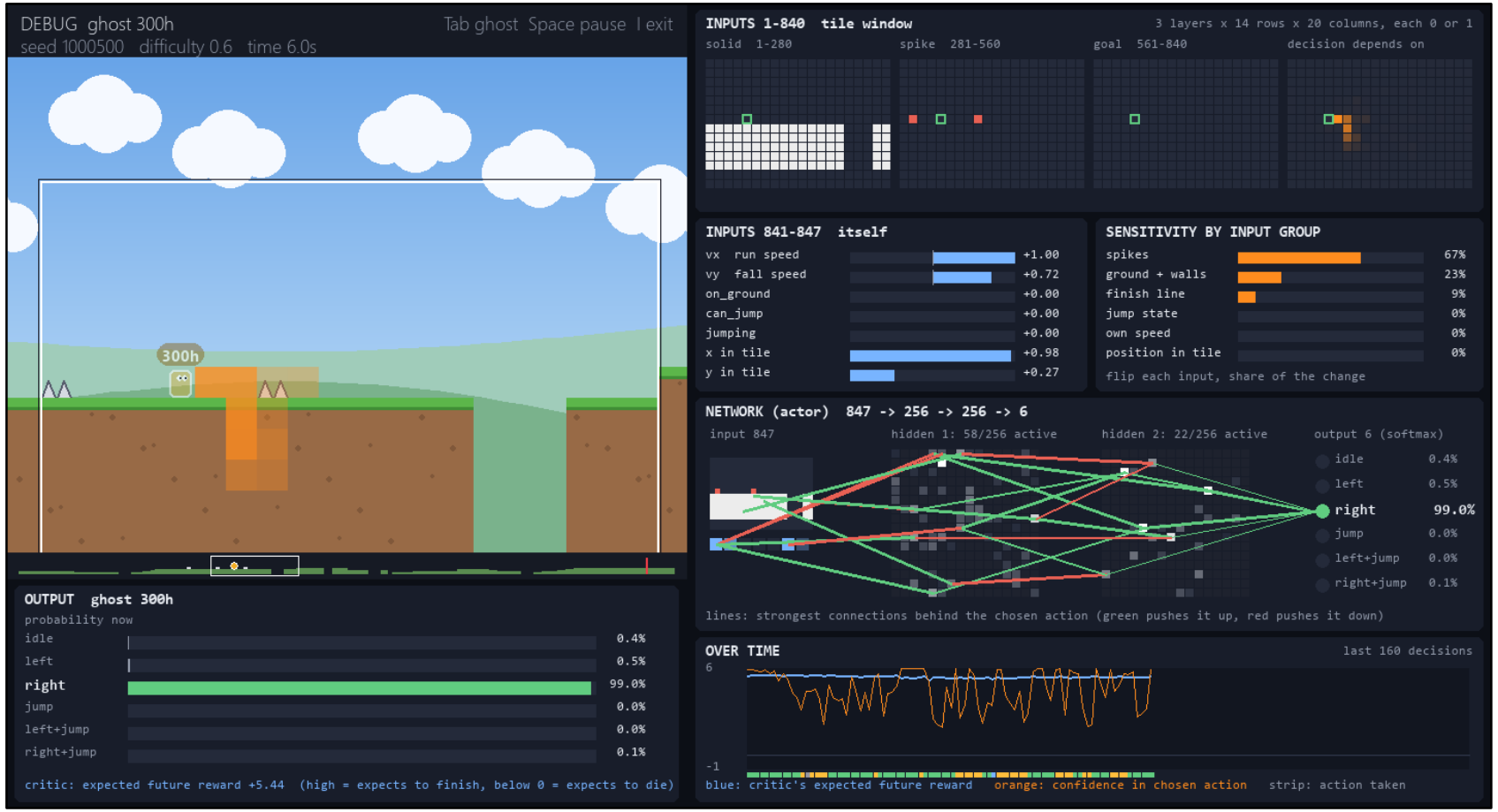
most learning happens in the first 30 hours

> cd section_05

05

OPENING THE BLACK BOX

DEBUG MODE: ONE GHOST, FULLY VISIBLE



- **view window** on the level; orange = tiles the decision depends on
- **840 tile inputs** as the 3 layers it receives
- **7 numbers** about itself
- **sensitivity** by input group, in %
- **live network:** every neuron, strongest links
- **6 probabilities:** the chosen one in green
- **critic** and confidence over time

```
$ python demo.py --debug 300h
```

I on/off Tab ghost Space pause Left/Right step O overlay F slow-mo

HOW WE ASK "WHAT MATTERED?"

> flip test (every decision, live)

1 take the 847 inputs
2 flip one: tile 0 <-> 1; speeds set to 0
3 run the network again
4 measure how far the 6 probabilities moved
  0 = not at all, 1 = a different decision
5 all 847 at once: one batched pass, ~7 ms

gives: the orange tiles, the % per group

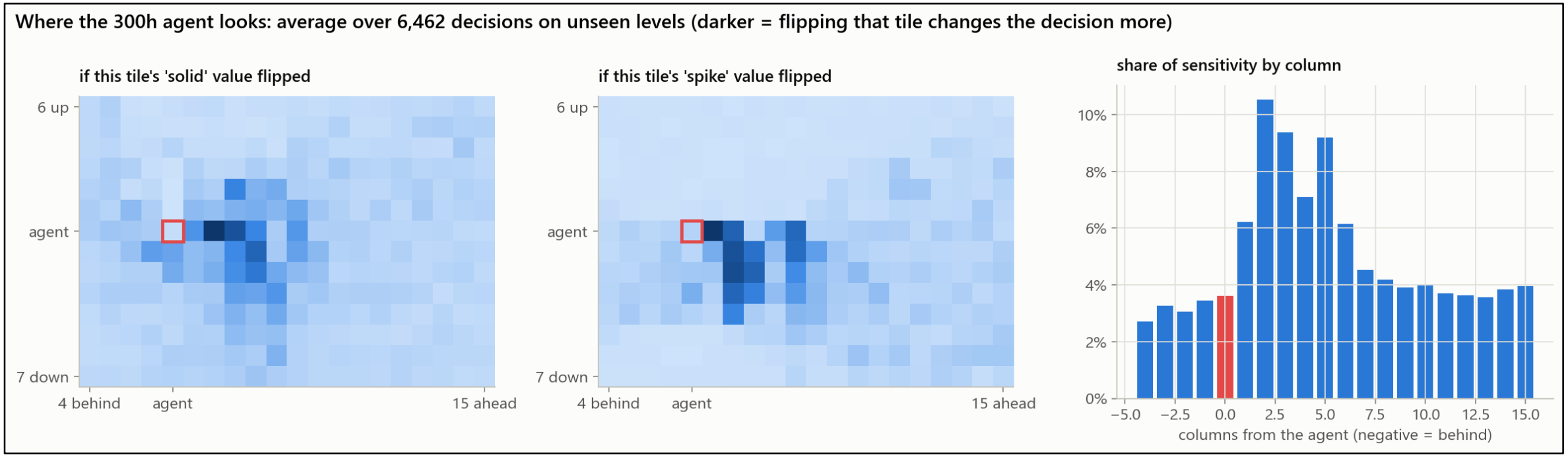
> blindfold test (whole levels)

1 remove one group of inputs for a whole
  run (set it to 0)
2 play 100 levels it has never seen
3 count how many it still finishes

gives: what it really relies on

sensitive to != relies on: "what if this were different?" vs "can it cope without it?"

WHERE IT LOOKS



2 tiles ahead

the column it watches most

49%

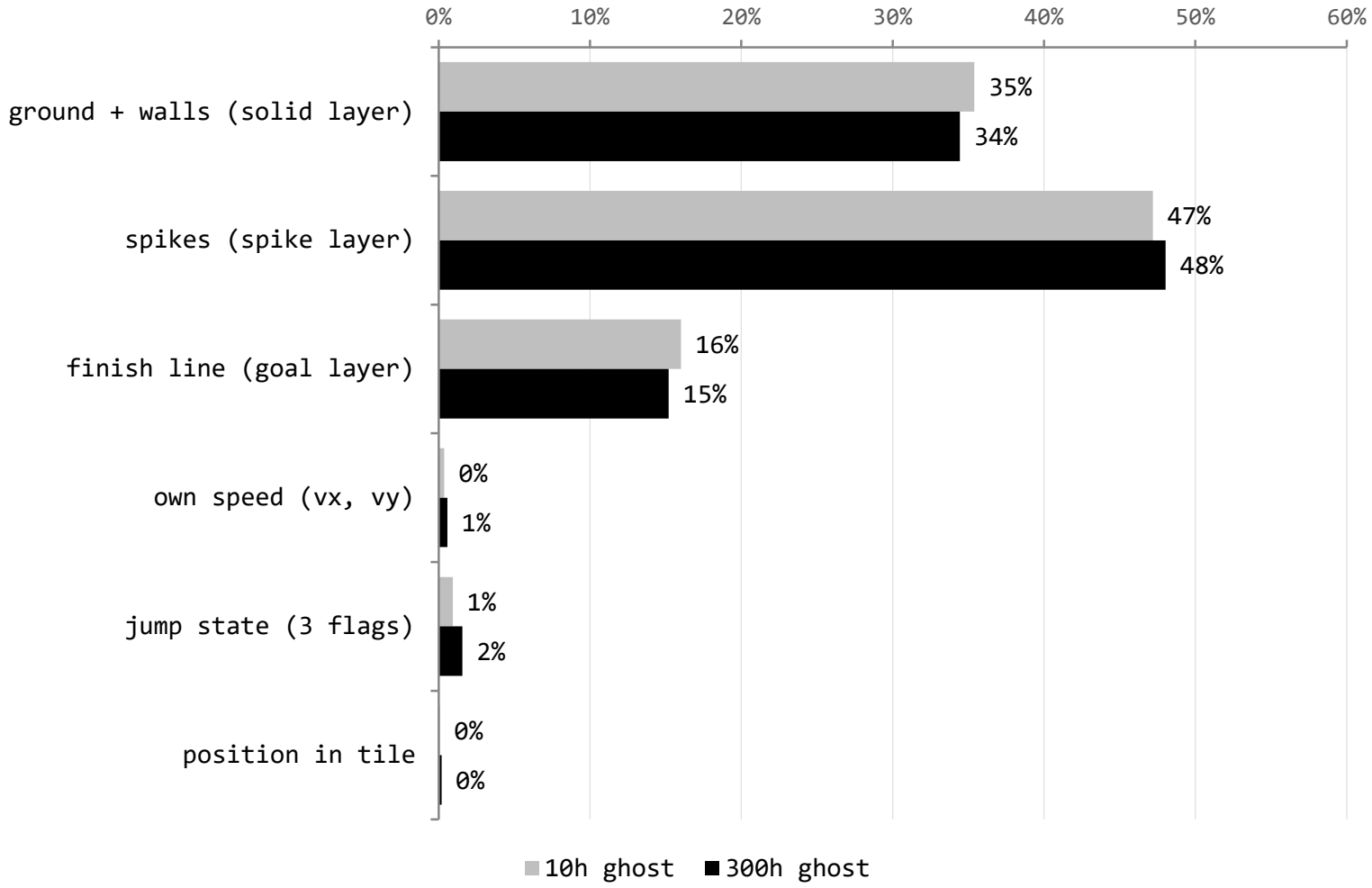
of its sensitivity is 1 to 6 tiles ahead

13%

on the 4 columns behind it

average over 6,462 decisions of the 300h ghost on 25 unseen levels, difficulty 0.75

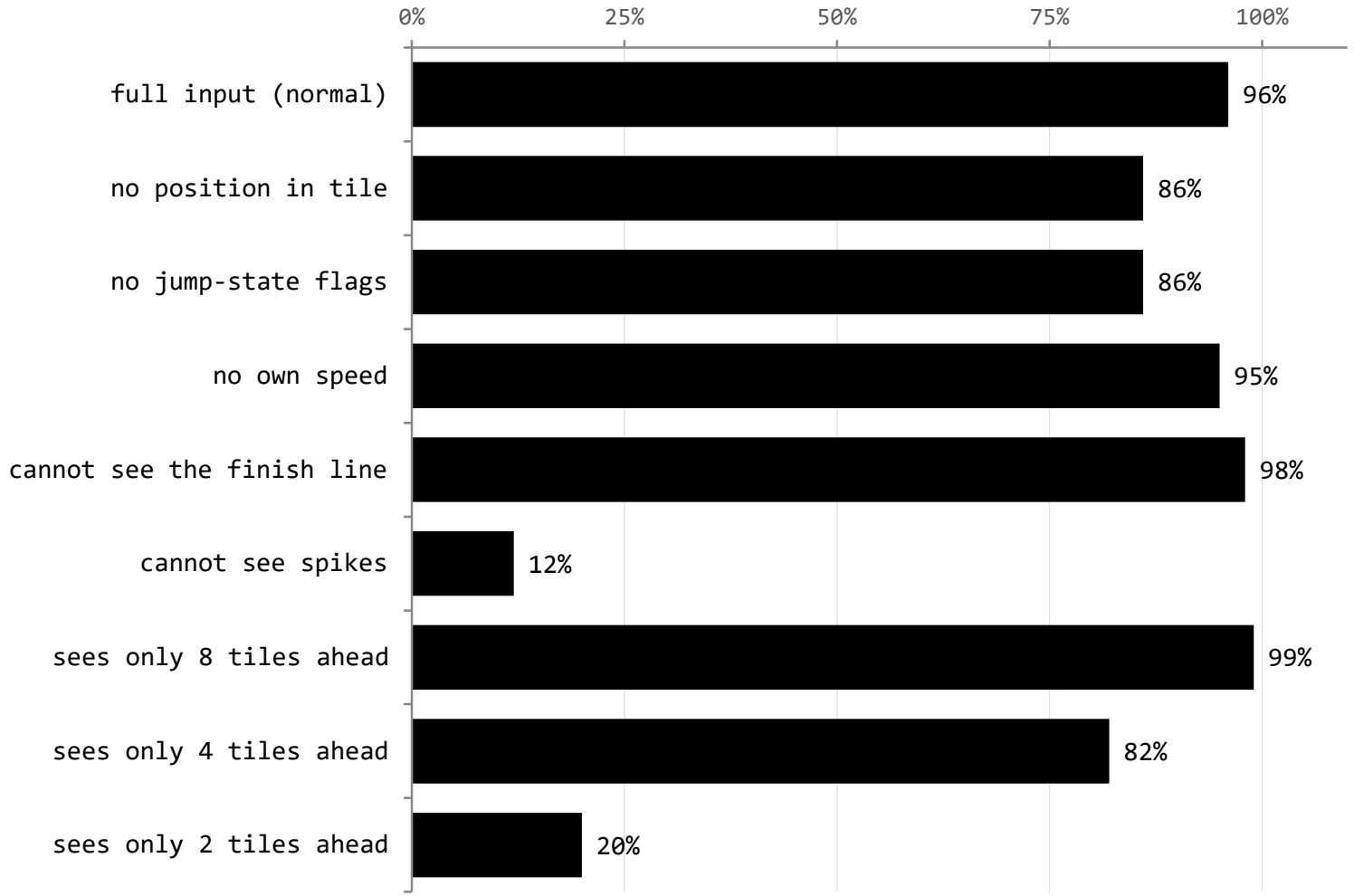
WHAT IT IS SENSITIVE TO



share of the total flip-test change per input group, averaged over all decisions on unseen levels

- **spikes 48%:** a spike appearing ahead would change its mind most
- **ground 34%:** where the ground starts and ends
- **finish line 15%**
- **its own state ~2%:** speed and jump flags matter only in the moment before a jump

BLINDFOLD TEST

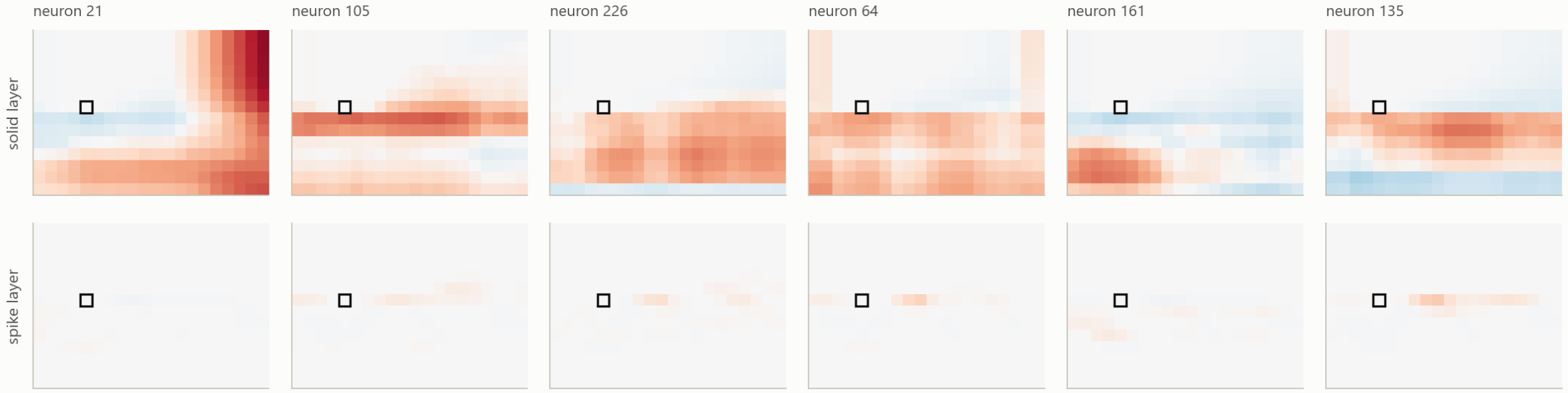


- **spikes are essential:** 96% -> 12%
- **8 tiles of look-ahead are enough** (99%); 2 tiles are not (20%)
- **jump flags, position in tile:** -10 points each
- **own speed, finish line:** hardly needed

300h ghost, 100 unseen levels at difficulty 0.75; removed inputs read as 0 (empty air, standing still)

NEURONS LEARN DETECTORS

What single neurons in hidden layer 1 respond to: the average view in their 5% most active moments, minus the average view
(red = more ground/spike here than usual, blue = less, e.g. a gap; box = agent)

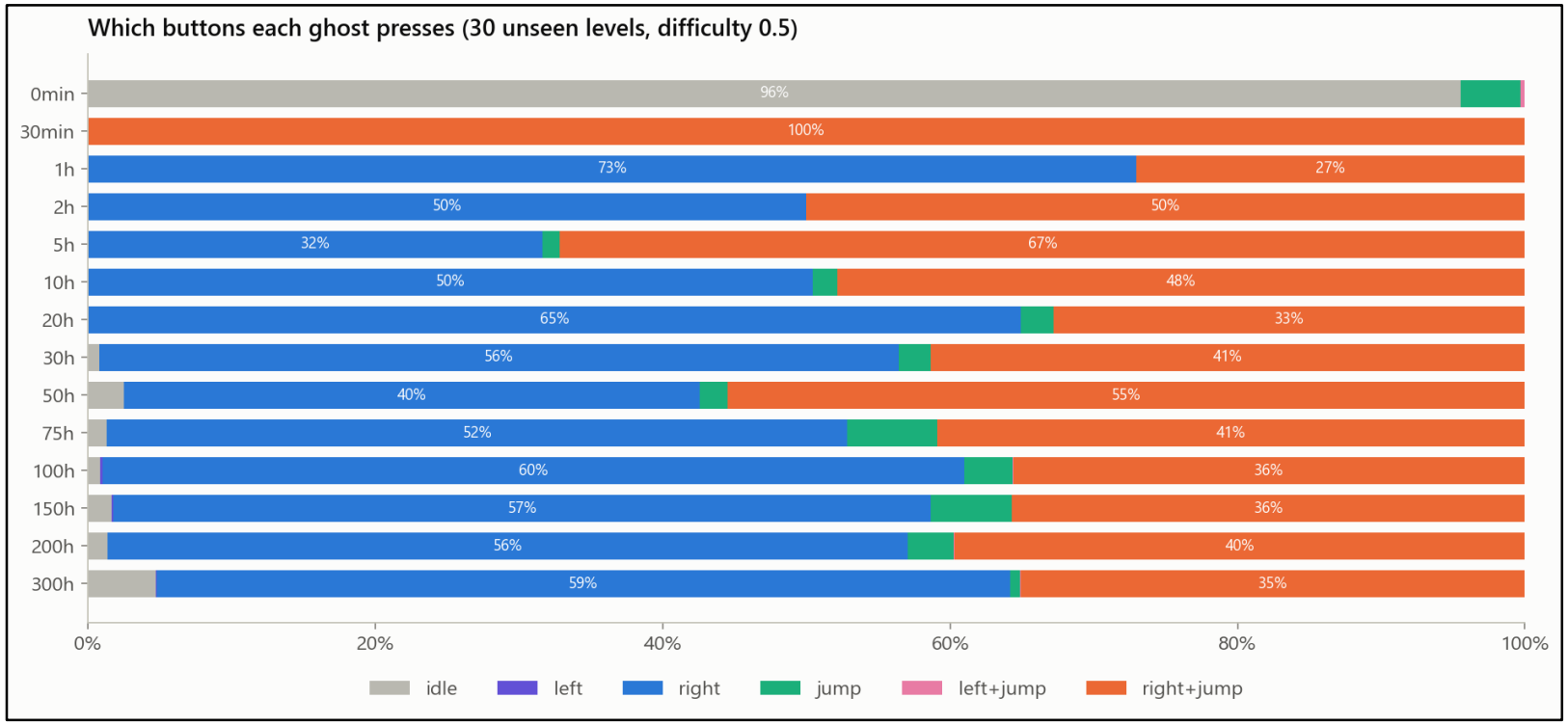


> **neuron 21:** the wall at the end of the level
> **neuron 105:** flat ground under its feet
> **neuron 161:** no ground ahead at foot level
> **neuron 135:** a step up ahead

> fewer, sharper neurons

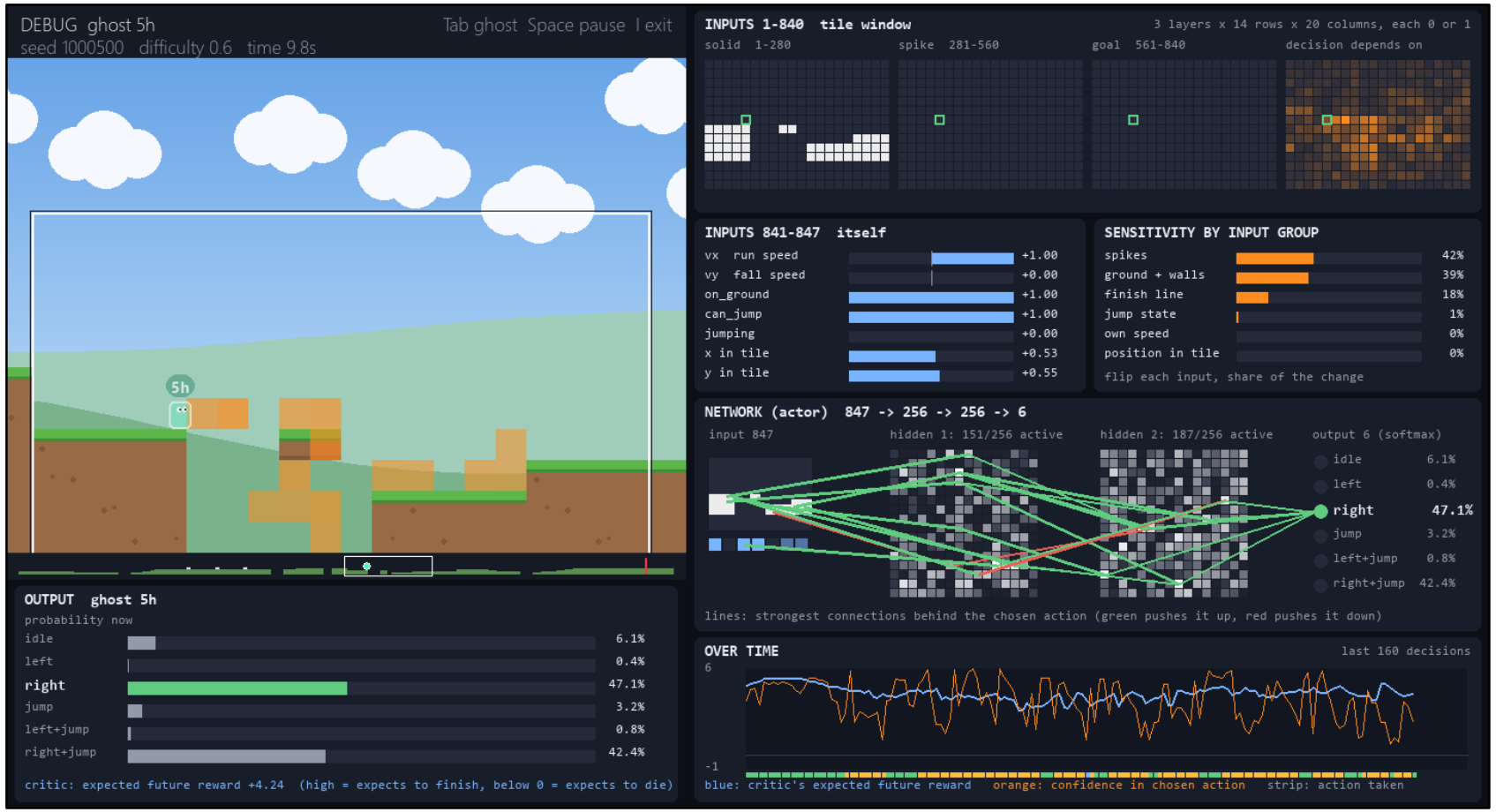
Active per decision (layer 1 / 2): untrained 138 / 135, 30h 93 / 110, 300h 66 / 30. Training made it specialised.

WHAT IT LEARNED TO PRESS



- **0 min:** idle 96%; random weights happen to favour idle
- **30 min:** right+jump 100%; hop over everything
- **300 h:** right 59%, right+jump 35%, idle 5%; jumps only when needed
- **left is never used:** nobody told it which way to go

A FAILURE, EXPLAINED



- **5h ghost at the edge of a pit, 9.8 s in**
- **undecided:** right 47% vs right+jump 42%
- **busy network:** 151 of 256 neurons fire; the 300h ghost needs about 60
- **result:** it runs off the edge; its critic falls from 5.3 to 0.5

same level, same moment: the 300h ghost is 99% sure and jumps in time

> cd section_06

06

RESULTS

ONE NIGHT ON A LAPTOP

> main run

300 h

of simulated play:
16.2 million decisions

> main run

4.7 h

of real training time,
CPU only

> main run

96%

of never-seen levels
finished at difficulty
0.75

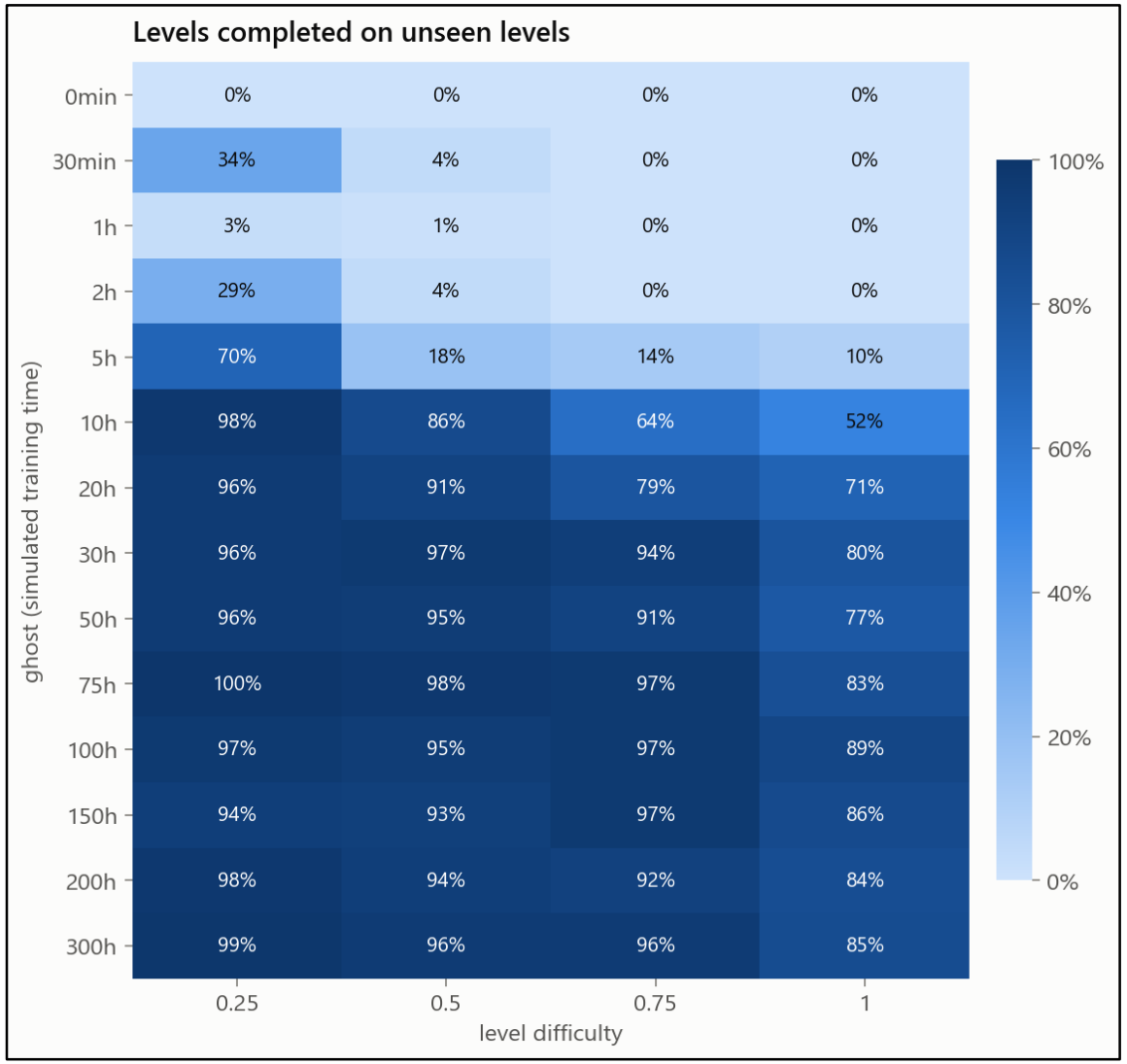
> main run

85%

at the hardest training
difficulty, 1.0

every test level comes from a seed range never used in training

EVERY GHOST AT EVERY DIFFICULTY

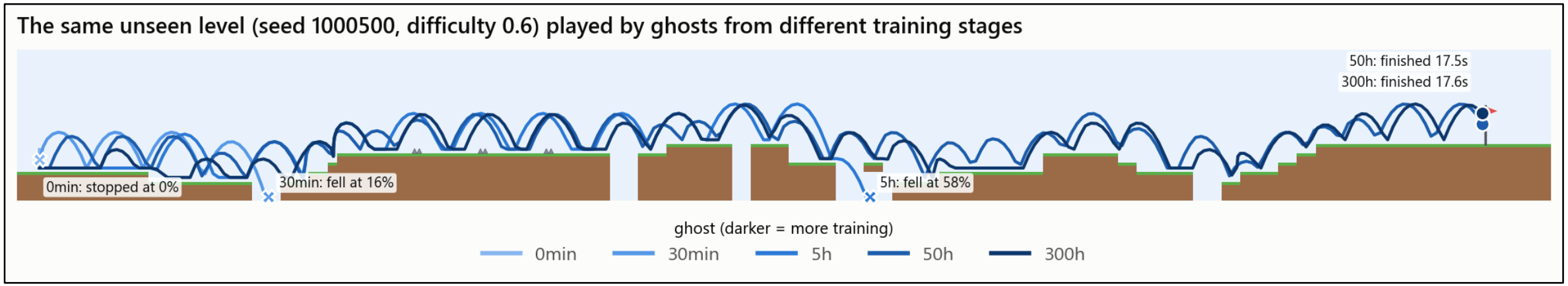


> **Across:** harder levels need more practice. At 10 h: 98% of easy levels, 52% of the hardest.

> **Down:** fast, then flat. 30 h to 300 h, ten times more training: 80% -> 85% on the hardest.

> **Top rows make the demo fun:** ghosts up to 2 h almost never finish a medium level.

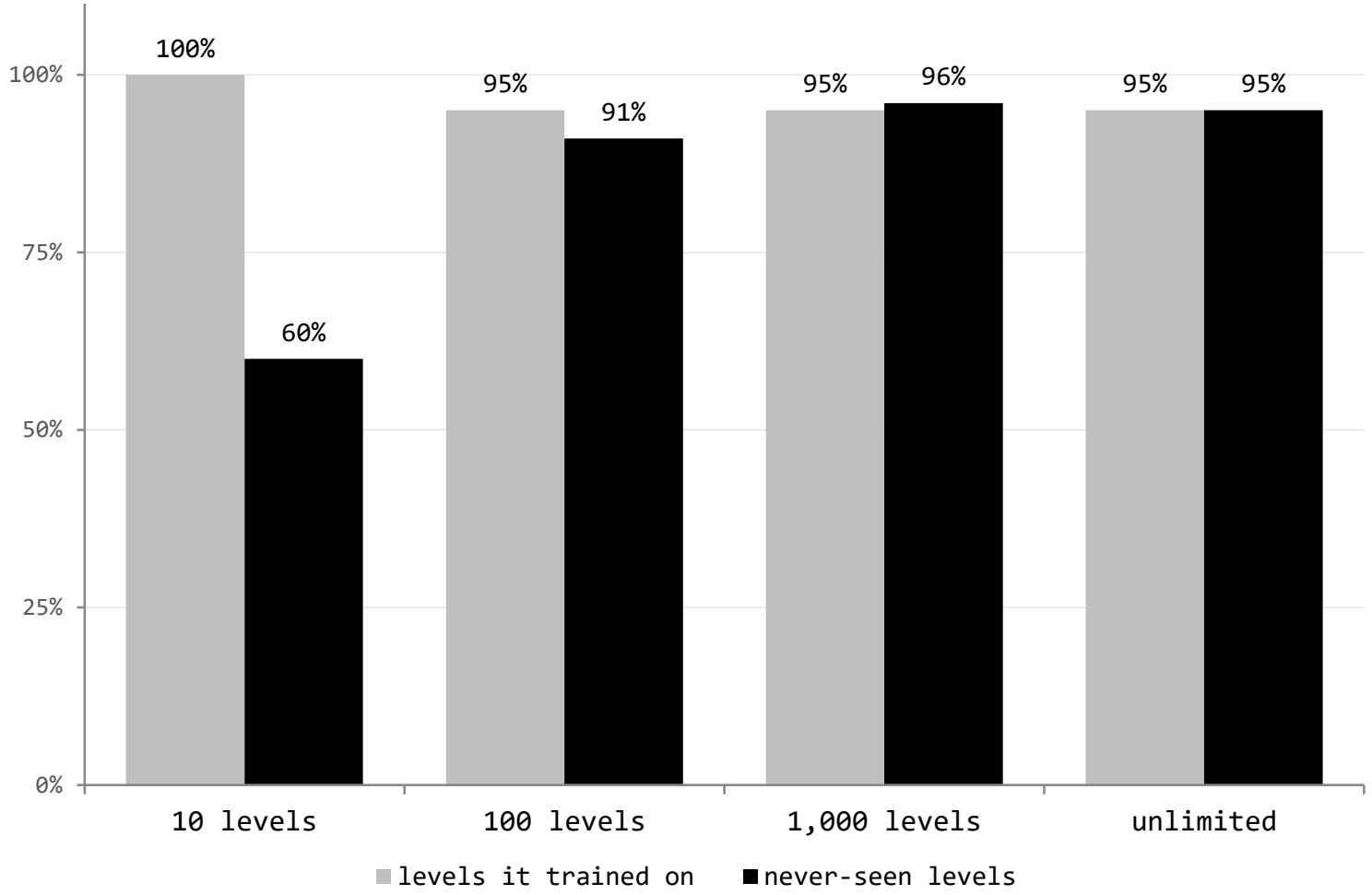
ONE UNSEEN LEVEL, FIVE STAGES



> 0 min	> 30 min	> 5 h	> 50 h	> 300 h
barely moves: 0%	first pit, 16%	a later pit, 58%	finishes, 17.5 s	finishes, 17.6 s

training buys survival, not speed: every ghost that finishes takes about 17 s

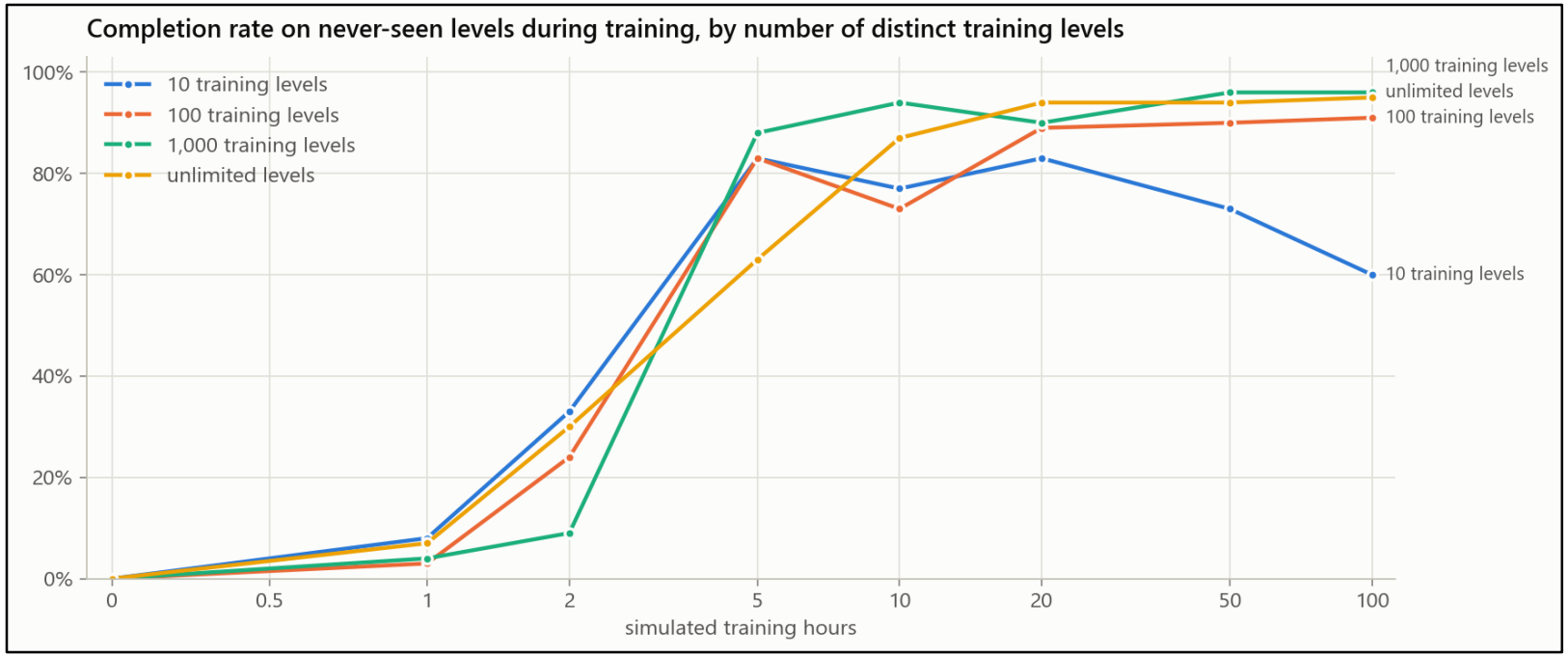
GENERALISATION: MEMORISE OR LEARN?



- **Same agent, same 100 h, same difficulty 0.5.** Only the number of different training levels changes.
- **10 levels:** 100% on those, 60% on new ones. It memorised them.
- **1,000+ levels:** no gap. It learned the game.

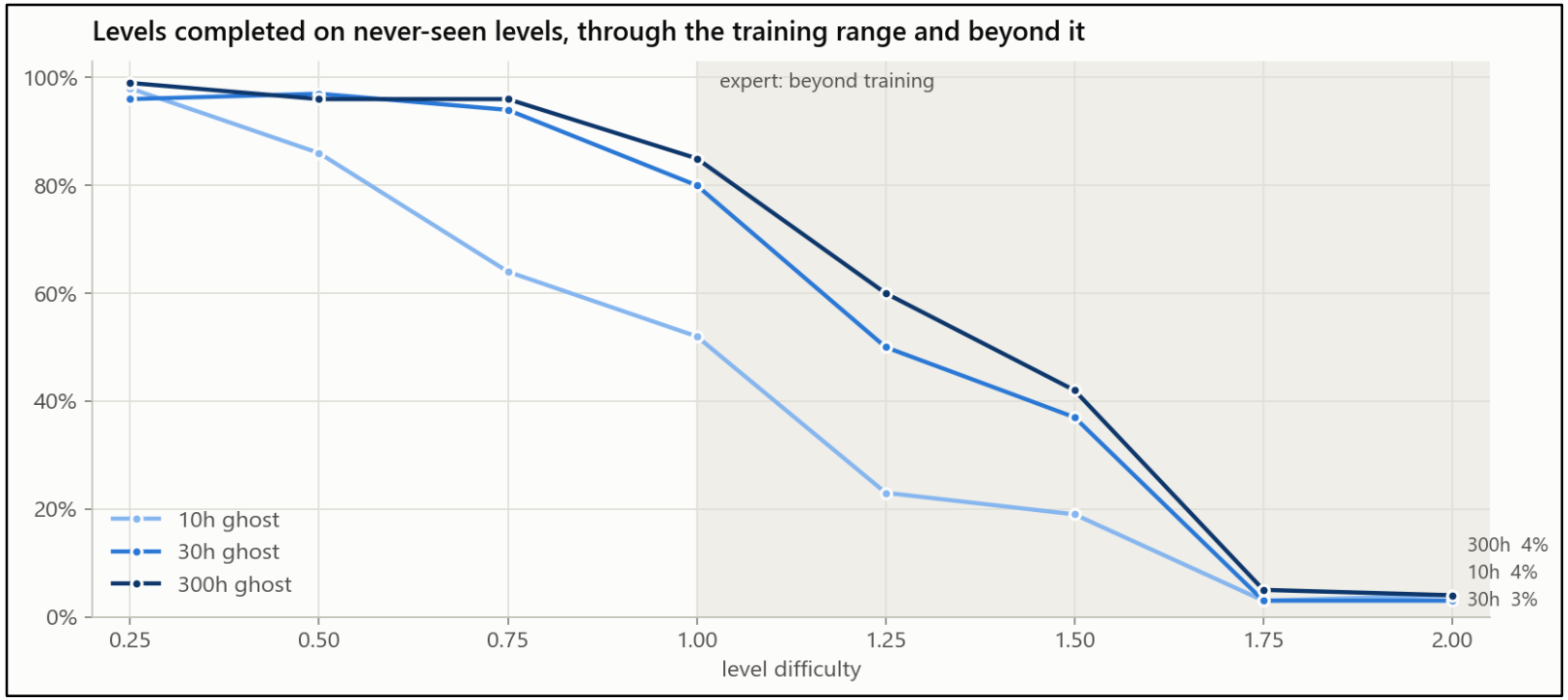
matches the Procgen benchmark finding (Cobbe et al., 2019)

MEMORISING GETS WORSE WITH TIME



- **10 levels: up, then down.** 83% on new levels at 5 h, 60% at 100 h.
- **More training made it worse** at the real game: overfitting, live.
- **Variety is not enough:** at 100 h the curriculum agent finishes 89% of the hardest levels, the one trained only at 0.5 finishes 64%.

EXPERT LEVELS: BEYOND TRAINING



> **Difficulty 1 to 2:** gaps up to 6 tiles, spike runs of 4, one-tile pillars, less rest

> **Same tiles, same physics,** no retraining; the solver checks every level

> **300h ghost:** 85% at 1.0, 60% at 1.25, 42% at 1.5, 5% at 1.75

> cd section_07

07

DEMO

RACE THE GHOSTS



> controls

arrows / A D	run
Space	jump
N	new level
[ ]	difficulty
I	debug mode
S	deterministic / random
H	all keys

```
$ python demo.py
```

```
$ python demo.py --debug
```

14 see-through ghosts from 0 min to 300 h; you are the solid red one; every round is a fresh level

WRAP-UP

> what worked

a tile view relative to the agent

a curriculum from easy to hard

a new level every attempt

one shared game loop: ghosts act exactly as in training

> what was hard

guaranteeing every random level is beatable (a solver in the tests)

a reward it can learn from before ever reaching a flag

overnight runs that survive a crash (autosave + resume)

> next

phase 2: enemies

pixel input with a convolutional network

a stronger critic (it is the weakest part)

compare human runs with the ghosts

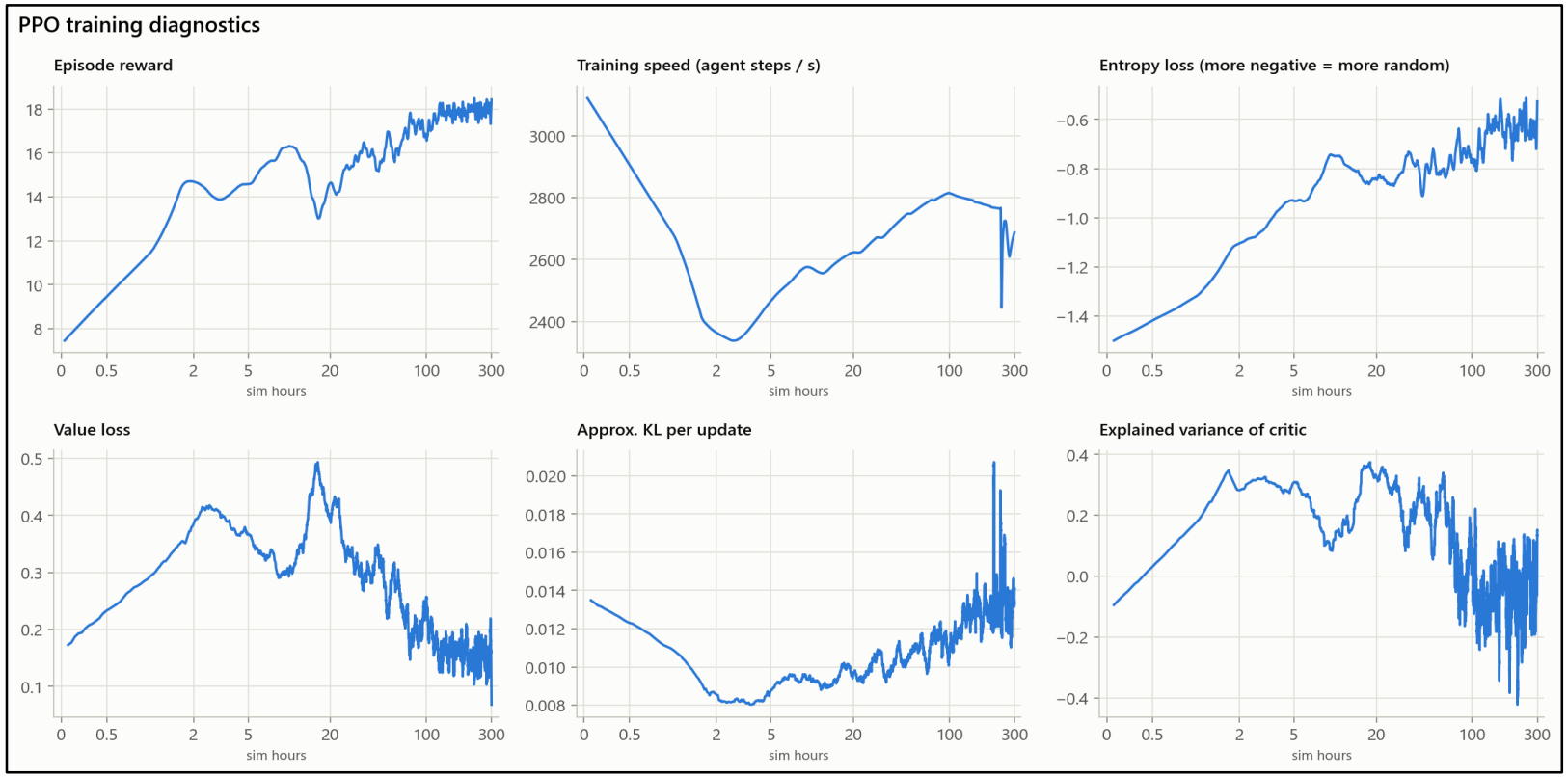
not a black box: it watches 2-6 tiles ahead, relies on spikes and ground, and built its own detectors

$ talk --questions

QUESTIONS?

[Your Name] // github.com/DYNAMAXD/ProcGen_RL

BACKUP: PPO HEALTH CHECKS



- **reward** rises from about 7 to 18 per attempt
- **entropy** falls slowly: fewer random actions
- **KL** stays near 0.01: stable updates
- **explained variance** below 0.4: the critic is the weak part

BACKUP: GLOSSARY

term	meaning here
reinforcement learning	learning from rewards by trial and error, without examples of correct play
policy / actor	the network that maps the 847 inputs to 6 action probabilities
critic / value	the network that predicts how much reward is still to come
advantage	how much better an action turned out than the critic expected
PPO	the training algorithm: small, clipped updates of the policy
episode	one attempt at one level, until death, the flag or a time-out
ghost	a saved copy of the policy after some amount of training
sim hours	hours of gameplay practised (decisions / 15 per second)
generalisation	doing well on levels never seen during training
overfitting	doing well on training levels by memorising them, badly on new ones